

Al-Nahrain University
College of Engineering
Department of Computer
Fourth Stage/First Semester



Microcontrollers Lab

Experiment No.4: Using 80C51 MCU Timers as a Delay Counter with
7-Segment Display

Course Lecture : Mustafa Adnan

Laboratory Assistant: Sarah Haider

Name of student: Aisha Ammar Abdalrhman

Date: 2025/

Objective :

1. To understand and utilize the 80C51 microcontroller timers for creating time delays.
2. To use the timer to create a counter that increments every specified delay interval.
3. To display the counter value on a 7-segment digital display.

Components Required :

- ❖ 80C51 Microcontroller Development Board
- ❖ Common Cathode 7-Segment Display
- ❖ Keil C Compiler
- ❖ Connecting Wires
- ❖ Breadboard (if necessary)
- ❖ Power supply

U1

19 XTAL1

18 XTAL2

9 RST

29 PSEN

30 ALE

31 EA

P0.0/AD0 39

P0.1/AD1 38

P0.2/AD2 37

P0.3/AD3 36

P0.4/AD4 35

P0.5/AD5 34

P0.6/AD6 33

P0.7/AD7 32

P2.0/A8 21 P20

P2.1/A9 22 P21

P2.2/A10 23 P22

P2.3/A11 24 P23

P2.4/A12 25 P24

P2.5/A13 26 P25

P2.6/A14 27 P26

P2.7/A15 28 P27

P10 1 P1.0

P11 2 P1.1

P12 3 P1.2

P13 4 P1.3

P14 5 P1.4

P15 6 P1.5

P16 7 P1.6

P17 8 P1.7

P3.0/RXD 10

P3.1/TXD 11

P3.2/INT0 12

P3.3/INT1 13

P3.4/T0 14

P3.5/T1 15

P3.6/WR 16

P3.7/RD 17

80C51

<TEXT>

P10 P11 P12 P13 P14 P15 P16

P20 P21 P22 P23 P24 P25 P26

Procedure:

1. The circuit was set up according to the circuit diagram.
2. Timer 0 was configured in Mode 1 to generate the required delay.
The timer registers (TH0 and TL0) were loaded with values corresponding to the desired delay.
3. The 7SEG-DIGITAL device was added from the Devices P library to the project, and the provided C program was loaded into the Keil IDE.
4. Two 7SEG-DIGITAL displays were connected to MCU ports P1 and P2 via a bus.
5. The MCU default clock was set to 12 MHz.
6. A suitable C program was written to configure the MCU and to display counting from 00 to 59 on the seven segments, with a 1-second delay between each count.
7. Algorithm:
 1. Timer 0 was initialized and was set to Mode 1.
 2. The Timer registers TH0 and TL0 were loaded with values to create the desired delay.

3. The timer was started by setting the TR0 bit in TCON.
4. The program was made to wait for the TF0 (Timer Overflow Flag) to be set. Once the overflow occurred:
 1. A counter variable was incremented.
 2. The TF0 flag was cleared.
 3. The 7-segment display was updated with the new counter value.
 4. The Timer registers TH0 and TL0 were reloaded with the initial delay values.
5. The process was repeated until the maximum count was reached or based on user control.

Theory:

Seven-segment digital in proteus is a common cathode configuration. Thus, we can add a code array for each display character. Since MCU clock 12MHz thus one count is 1us and we need 1,000,000 count to be 1 sec. Maximum counts is 64k count thus we need to divide this number for example 50,000 count each time which is 50ms and repeat it 20 times will result in 1000ms >> 1s

Things change for common cathode configuration.

Number	g f e d c b a	Hex Code
0	0111111	3F
1	0000110	06
2	1011011	5B
3	1001111	4F
4	1100110	66
5	1101101	6D
6	1111101	7D
7	0000111	07
8	1111111	7F
9	1001111	4F

Code:

```
#include <reg51.h>

#include <stdio.h>

// 7-segment display codes for digits 0-9 (common cathode)

code unsigned char Seg7[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D,
0x7D, 0x07, 0x7F, 0x6F};

void T0Delay(void);

void Delay1sec(void);

// Main program

void main(void)
{
    unsigned char tens = 0;    // Tens digit (0-5)
    unsigned char ones = 0;    // Ones digit (0-9)

    while(1)
    {
        // Outer loop for tens digit (0 to 5)
        for(tens = 0; tens < 6; tens++)
        {
            // Inner loop for ones digit (0 to 9)
            for(ones = 0; ones < 10; ones++)
```

```

    {
        // Display on 7-segment displays

        P1 = Seg7[tens]; // Display tens digit on P2

        P2 = Seg7[ones]; // Display ones digit on P1


        // Wait for 1 second

        Delay1sec();

    }

}

// After reaching 59, loop automatically restarts from 00

}

}

// Function to generate 50ms delay using Timer 0

void T0Delay(void)

{
    TMOD = 0x01; // Timer 0, Mode 1 (16-bit timer)

    TL0 = 0xB0; // Load lower byte for 50ms delay

    TH0 = 0x3C; // Load higher byte for 50ms delay

    TR0 = 1; // Start Timer 0

    while(TF0 == 0); // Wait until timer overflows

    TR0 = 0; // Stop Timer 0

    TF0 = 0; // Clear overflow flag

}

```

```
// Function to generate 1 second delay (20 x 50ms = 1000ms)
```

```
void Delay1sec(void)
```

```
{
```

```
    unsigned char i;
```

```
    for(i = 0; i < 20; i++)
```

```
    {
```

```
        T0Delay();
```

```
    }
```

```
}
```

Discussion Questions :

1. Why do we use Timer 0 in Mode 1 for generating a delay in this experiment?

Timer 0 is used in Mode 1 because this mode configures the timer as a 16-bit timer, allowing it to count from 0000H to FFFFH (0–65535).

This provides a larger counting range, which is necessary for generating long delays such as 1 second.

Other modes (like Mode 0 or Mode 2) have smaller count ranges and cannot produce the required delay without complex handling.

2. How do we calculate the initial values to load into TH0 and TL0 for a specific delay?

The calculation is based on:

- MCU clock = 12 MHz
- 1 machine cycle = 12 clock cycles → 1 μ s per cycle
- Timer increments every 1 μ s

Steps to calculate TH0/TL0:

1. Determine the required delay (for example 50,000 μ s = 50 ms).

2. Find how many counts timer must run:

Counts needed = Delay in μ s
 $\text{Counts needed} = \frac{\text{Delay in } \mu\text{s}}{1 \mu\text{s per cycle}}$

3. Since maximum count = 65,536 (for 16-bit timer), load:
Initial value = 65536 - Counts needed
 $\text{Initial value} = 65536 - \text{Counts needed}$
4. Convert the result to hex, then split into:
 - High byte → TH0
 - Low byte → TL0

Example (50 ms):

- 50 ms = 50,000 μ s
- Initial value = 65,536 - 50,000 = 15,536
- 15,536 → 0x3CB0
- TH0 = 0x3C, TL0 = 0xB0

3. What would happen if we did not reload the Timer registers (TH0 and TL0) after each overflow?

If we do not reload TH0 and TL0:

- The timer would start counting from 0000H after each overflow.
- This would give a much longer delay than intended (full 65,536 μ s).
- The delay would no longer match the required 50 ms (or whatever delay was calculated).
- The 7-segment counting would become incorrect, slower, and inconsistent.

So reloading ensures accurate and stable timing.

4. Why is it necessary to clear the Timer Overflow Flag (TF0) after each timer overflow?

TF0 is automatically set to 1 when the timer overflows.

If TF0 is not cleared:

- The MCU will think the timer has already overflowed again.
- The next delay cycle will not wait for a real overflow.
- This causes incorrect timing and immediate repeated execution.

Therefore, TF0 must be cleared to allow the timer to function correctly for the next delay.

5. How can we modify this experiment to count down instead of counting up on the 7-segment display?

To convert the program to count down:

1. Initialize the counter with 59 instead of 0.
2. In the overflow routine, replace:
3. When the counter reaches 00, reset it to 59.
4. The 7-segment display update function will show the decreasing value.

Conclusion :

This experiment demonstrates how to use the 80C51 MCU timers to generate a delay, acting as a counter. The counter values are effectively displayed on a 7-segment display, reinforcing concepts of timer handling and digital display interfacing.